Review 1: what to say

Pharmacy Inventory & Prescription Management · BCSE302P · Sathyaraj · about 6 minutes spoken, 6 to 8 minutes with clicking

The gist (if you remember one thing)

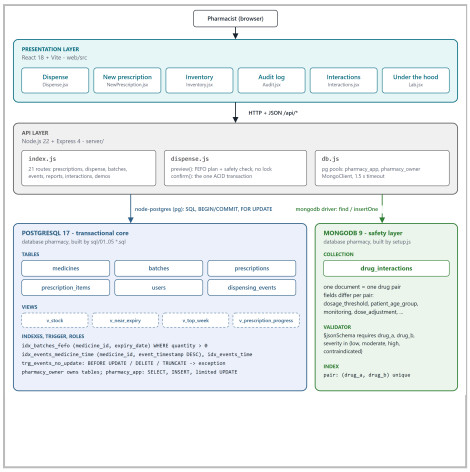
> We built a pharmacy dispensing system on two databases. PostgreSQL keeps stock, prescriptions and an audit log that cannot be edited, and runs every dispense as one short ACID transaction with row locks. MongoDB keeps drug-interaction rules whose fields differ per drug pair, and is checked before and inside every dispense; if it is down, nothing is dispensed.

Nothing is split between speakers: the faculty can ask any of us anything, so each of us learns the whole script and all the answers. Say it in your own words; the lines below are what has to come across.

Numbers to know

What	Value
Medicines / live batches / all batches	44 / 181 / 60,181 (60,000 emptied history batches so indexes matter)
Prescriptions / audit events at start	10 / 3,181
Interaction documents / distinct field names	30 / 18
API routes / end-to-end checks	21 / 24 of 24 pass
FEFO query with index vs without	0.037 ms vs 7.205 ms, about 195x; 3 buffers vs 443
Race: B waits	from 258 ms to 2,019 ms, locks 1 ms after A commits, takes next batch
MongoDB timeout	1.5 s, then HTTP 503 and ROLLBACK
Versions	PostgreSQL 17.11, MongoDB 9.0.2, Node 22, Express 4, React 18, Vite 6

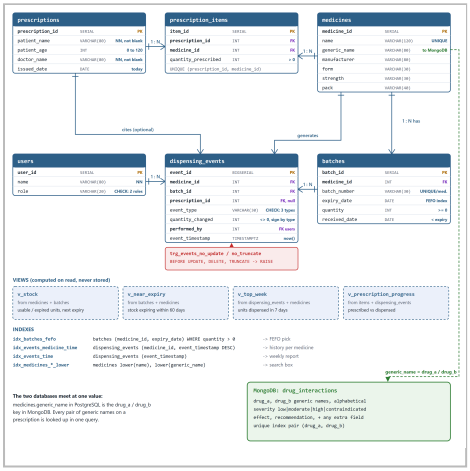
Part A: the diagrams (about 2 minutes)



Proposed model (Figure 3) 25 s · report page 8

Point: Point top to bottom: Pharmacist, React screens, API boxes, then PostgreSQL box, then MongoDB box.

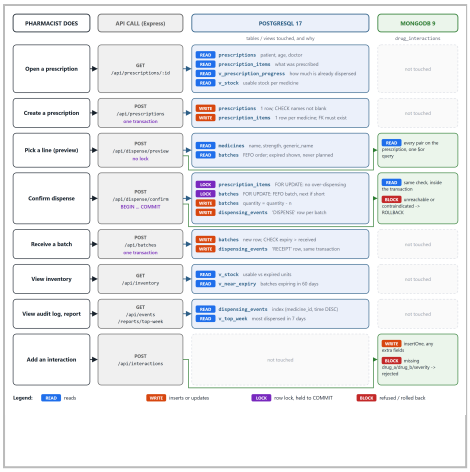
This is the full model. Three layers. The pharmacist only uses the browser, the React screens. The browser only calls our Express API. And only the API talks to the databases, so every rule is enforced on the server or in the database, never in the browser. At the bottom, the two engines: PostgreSQL is the transactional core with six tables, four views, the indexes, a trigger and two roles. MongoDB is the safety layer with one collection, drug_interactions, which has a validator and a unique index on the drug pair.



Complete data model (Figure 4) 40 s · report page 10

Point: medicines, batches (1:N), prescriptions to items, users, dispensing_events, the red trigger box, views row, then follow the green dashed line from generic_name to the MongoDB box.

These are the six tables. medicines is the product. batches is one physical lot of a medicine with its expiry and quantity, and batches is the source of truth for stock. prescriptions is the header, patient and doctor; prescription_items is one line per medicine with the quantity prescribed, which resolves the many-to-many between prescriptions and medicines. users are the pharmacists. dispensing_events is the audit log: every dispense, receipt and adjustment, which batch, how much, who and when. It is append-only; this red trigger rejects any UPDATE, DELETE or TRUNCATE. The dashed views compute stock and reports on read. And the two databases meet at one value: medicines.generic_name is the drug_a / drug_b key in MongoDB.



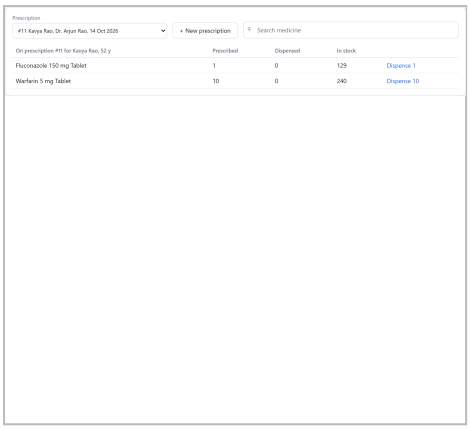
Data flow, what goes where (Figure 5) 55 s · report page 12

> Point: Run your finger down the rows. Stop on "Confirm dispense": purple LOCK, orange WRITE, the green arrow to MongoDB, red BLOCK.

This answers what data goes where. Each row is one thing the pharmacist does. Blue is a read, orange is a write, purple is a row lock. Opening a prescription and previewing a dispense only read, so nothing is locked while the pharmacist reads a warning. Previewing also asks MongoDB for every pair of medicines on the prescription in one query. Only Confirm locks: first the prescription line so it cannot be over-dispensed, then the batch in expiry order. It checks MongoDB again inside the transaction, then updates the batch and writes a DISPENSE row in the audit log, and commits both together. Receiving a batch writes the batch and a RECEIPT row in one transaction too. So every stock change leaves an audit row, and the audit log adds up exactly to the stock. Adding an interaction only touches MongoDB, and a document missing the required fields is rejected.

If he asks how a dispense works step by step, open Figure 9 (page 21): "Preview reads both databases and locks nothing. Confirm is one transaction: lock the prescription line, lock the batch by expiry, check MongoDB again, update, write the audit row, commit. Any failure, the red box, is a rollback."

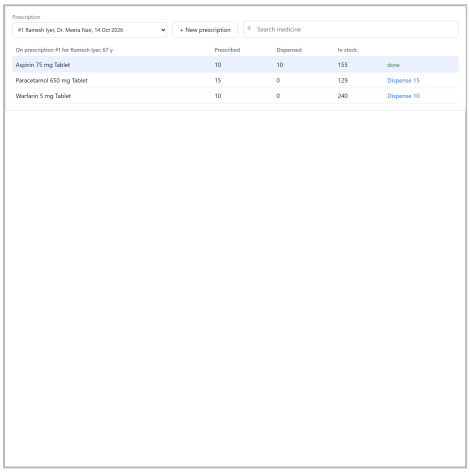
Part B: the demo (about 5 minutes)



Step 1 · Create a prescription 30 s · fallback: Figure 12, 13

> Do: Dispense tab, "+ New prescription". Type Kavya Rao, 52, Dr. Arjun Rao. Warfarin 5 mg, 10. "+ Add medicine", Fluconazole 150 mg, 1. Save prescription.

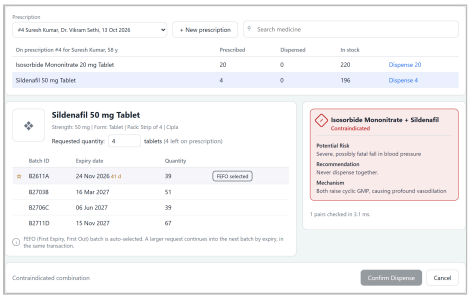
Both lights in the header are green: PostgreSQL and MongoDB are reachable. I am creating a new prescription. Saving inserts one row in prescriptions and one row per medicine in prescription_items, in a single transaction; a blank name is refused by a CHECK constraint and nothing is half-written.



Step 2 · Dispense with a warning 60 s · fallback: Figure 14, 15, 16, 17

> Do: Click "Dispense 1" on Fluconazole: warning appears. Then choose prescription #1 Ramesh Iyer, click "Dispense 10" on Aspirin. Tick "I have reviewed this warning". Confirm Dispense.

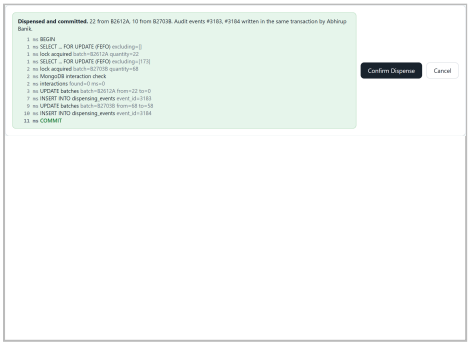
Picking a line runs the preview, with no lock. MongoDB immediately finds Fluconazole with Warfarin on the new prescription. On prescription 1, Aspirin: the batches are in expiry order, the expired batch is struck through and never offered, and the earliest unexpired batch is starred, that is FEFO. Aspirin with Warfarin is high severity, so Confirm is disabled until I acknowledge it. Now Confirm. This trace is the real transaction: BEGIN, lock the prescription line, lock the batch, MongoDB check, UPDATE, INSERT into the audit log, COMMIT, in about 30 milliseconds. Stock went from 165 to 155.



Step 3 · Unsafe dispense refused 20 s · fallback: Figure 19

> Do: Choose prescription #4 Suresh Kumar, click "Dispense 4" on Sildenafil.

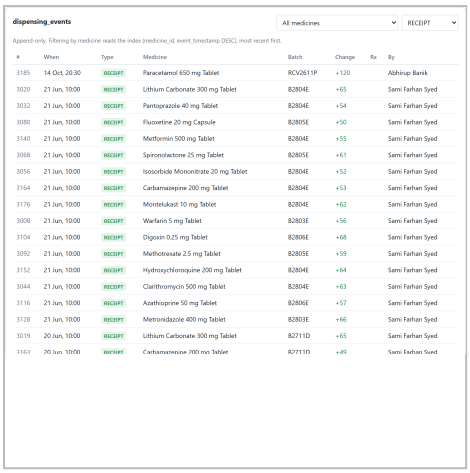
Sildenafil with Isosorbide is contraindicated, so there is no acknowledge box at all: it is blocked. Even if someone bypassed the button, the server refuses with 409 and rolls back.



Step 4 · Request larger than one batch 30 s · fallback: Figure 20, 21

> Do: Choose "Walk-in", search Montelukast, click it, set quantity to 32. Confirm Dispense.

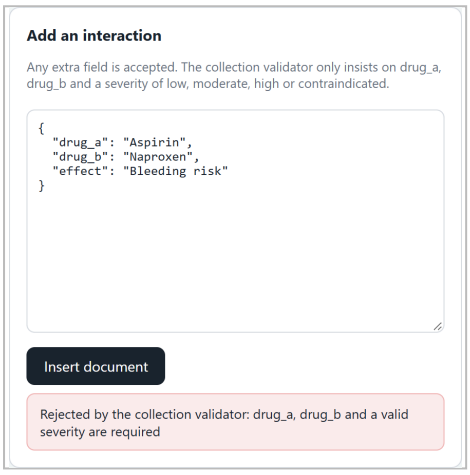
The FEFO batch has only 22. The plan takes 22 from the first batch and then 10 from the next by expiry. Confirm locks both batches in order and writes two audit rows, one per batch, in the same transaction.



Step 5 · Receive stock, inventory, audit log 40 s · fallback: Figure 22, 23, 25

Do: Inventory tab, click Paracetamol 650 mg. Batch number RCV2611P, expiry 31-03-2028, quantity 120, Receive. Then Audit log tab, set the type filter to RECEIPT.

Receiving writes the new batch and a RECEIPT row in one transaction; a CHECK rejects an expiry before the received date. The stock table comes from the v_stock view, with expired units counted separately. In the audit log, filtered to receipts, our batch is on top with who and when. The bars are the most dispensed medicines this week, from v_top_week.



Step 6 · MongoDB flexibility and its floor 30 s · fallback: Figure 28, 29

Do: Interactions tab. Insert document (the sample is pre-filled). Then replace the text with a document without severity and Insert again.

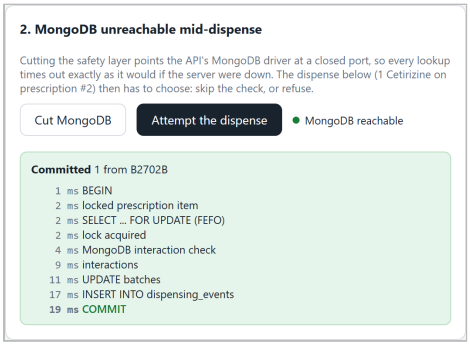
Each document has different fields. This new one has separate_by_hours and evidence_level, which no other document has, and it is accepted with no ALTER TABLE and no migration. But a document without a severity is rejected by the $jsonSchema validator, so flexible does not mean anything goes.



Step 7 · Two pharmacists, the last unit 40 s · fallback: Figure 30

Do: Under the hood tab, "Run the race". Wait about 2 seconds.

The oldest Cetirizine batch is set to exactly one unit. Pharmacist A locks it with SELECT FOR UPDATE and holds the lock for two seconds. B asks for the same unit 250 milliseconds later and blocks. Look at the times: B gets the lock one millisecond after A commits, sees the batch is now empty, and takes the next batch. Both succeed, and the unit is sold once.



Step 8 · MongoDB unreachable 30 s · fallback: Figure 31, 32

Do: "Cut MongoDB", wait for the red dot, "Attempt the dispense". Then "Restore MongoDB", wait for green, attempt again.

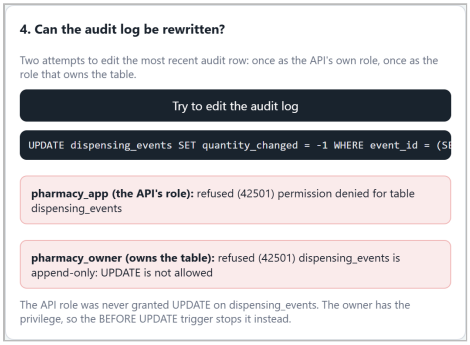
This points the driver at a closed port, exactly as if MongoDB were down. The dispense reaches the safety check inside the transaction, times out after 1.5 seconds, and rolls back. Restore, same dispense, it commits. We never dispense without the safety check.



Step 9 · Why the FEFO query stays fast 20 s · fallback: Figure 33

Do: "Run EXPLAIN ANALYZE".

Batches has over 60,000 rows but only 180 have stock. The partial index stores only those, so the plan is an index scan reading 3 buffers in hundredths of a millisecond; with index scans off it is a sequential scan, about 195 times slower.



Step 10 · The audit log cannot be rewritten 20 s · fallback: Figure 34

Do: "Try to edit the audit log".

The API role was never granted UPDATE on the audit table, so it fails on privilege, error 42501. The owner has the privilege, so the trigger stops it. The audit log is append-only by the database itself. That is the project.

Questions he may ask (any of us)

Design

Q. Why two databases and not one?
The data behaves differently. Stock and prescriptions need ACID transactions, row locks and constraints, which is PostgreSQL. Interaction rules have a different set of fields for each pair and are read on every dispense but rarely written, which suits a document store. That is polyglot persistence.

Q. Could the interactions live in PostgreSQL as JSONB?
Yes, technically. We kept them in MongoDB because they are curated knowledge that changes independently of stock, the validator gives a schema floor with free extra fields, and it shows the polyglot design the project is about. The cost is a second system to run, which is why the dispense blocks if it is down.

Q. Why did you drop Cassandra from Review 0?
The audit log is written in the same transaction as the stock change. Keeping it in PostgreSQL gives that atomicity for free; a third engine would need two-phase commit or could drift.

Q. Where do the two databases connect?
At medicines.generic_name. The API takes the generic names of every medicine on the prescription, forms all pairs, and queries drug_a / drug_b in MongoDB with one $or query.

Q. Does a dispense write to MongoDB?
No, it only reads it. All writes go to PostgreSQL in one transaction, so there is no cross-database commit problem: if PostgreSQL rolls back, nothing in MongoDB needs undoing.

Schema and normalisation

Q. Which normal form is the schema in?
Third normal form. Every non-key column depends on the whole key and only on the key: batch details on batch_id, product details on medicine_id. prescription_items resolves the many-to-many between prescriptions and medicines, with UNIQUE (prescription_id, medicine_id).

Q. Why is stock not a column in medicines?
Stock belongs to a batch, because each batch has its own expiry. Total stock is computed by the v_stock view as a sum over batches, so it can never disagree with the batches.

Q. Why keep the audit log if batches already has the quantity?
batches says what is on the shelf now; dispensing_events says how it got there, who and when. Because both are written in one transaction, the sum of the audit log equals the stock exactly; we checked it.

Q. What constraints did you use?
Primary and foreign keys; NOT NULL; CHECK quantity >= 0, expiry after received, age 0 to 120, names not blank, event type in three values, and the sign of quantity_changed must match the event type; UNIQUE on medicine name and on batch number per medicine.

Q. What does prescription_id being nullable mean?
Walk-in sales and stock receipts are not tied to a prescription, so their audit rows have no prescription.

Transactions and concurrency

Q. Explain ACID using your dispense.
Atomicity: the UPDATE and the audit INSERT commit together or roll back together. Consistency: CHECK and foreign keys stop invalid states, such as negative stock. Isolation: FOR UPDATE row locks serialise two pharmacists on the same batch. Durability: after COMMIT, PostgreSQL has written it to the WAL.

Q. What isolation level do you use?
PostgreSQL default, READ COMMITTED, plus explicit row locks with SELECT ... FOR UPDATE. That is enough because the conflict is on known rows.

Q. In the race, how does B end up on the next batch?
B blocks on A's row lock. When A commits, PostgreSQL re-checks B's WHERE condition against the new version of the row; quantity is now 0, so the row no longer qualifies and the scan moves on to the next batch in expiry order.

Q. Why not lock during the warning, like your Review 1 diagram?
A row lock held while a person reads would block every other pharmacist on that medicine for as long as they read. So preview takes no lock, and Confirm runs one short transaction that re-checks everything. That is the main change since the design review.

Q. Can two confirms deadlock?
Every transaction locks in the same order: the prescription line first, then batches in expiry order. With a single global order, a cycle of waits cannot form.

Q. Why not SKIP LOCKED?
SKIP LOCKED would let B take a later batch while the earliest one still has stock, which breaks FEFO. We prefer B to wait a few milliseconds.

Q. Why not SERIALIZABLE?
It would also work, but conflicts would surface as serialization failures that must be retried. Explicit row locks make the waiting visible and need no retry loop.

Q. What if the server crashes mid-dispense?
The transaction never committed, so PostgreSQL discards it on recovery. Stock and audit log stay consistent.

Q. How do you stop over-dispensing a prescription?
The prescription line is locked FOR UPDATE first, then the amount already dispensed is compared with what was prescribed; more than the remainder is refused with 409.

Indexes and queries

Q. Explain the FEFO index.
idx_batches_fefo on (medicine_id, expiry_date) WHERE quantity > 0. medicine_id is the equality, expiry_date gives the order, so the first index entry is the answer. It is partial: only batches with stock are stored, 180 of 60,182, so it stays small.

Q. Why does your FEFO query exclude expired batches?
expiry_date > current_date. In the wireframe the starred batch had already expired; selling it is wrong, so expired stock is shown struck through and counted separately.

Q. Show the speed difference.
EXPLAIN ANALYZE: 0.037 ms with the index against 7.205 ms with index scans off, about 195 times; 3 buffers against 443, and the sequential scan filters out 60,178 rows.

Q. Which other indexes, and why?
(medicine_id, event_timestamp DESC) for one medicine's recent history; event_timestamp for the weekly report; prescription_id on items; lower(name) and lower(generic_name) for search.

Q. Views or materialized views?
Plain views, computed on read, so they are always current. At this size they are instant; a materialized view would need refreshing after every dispense.

MongoDB

Q. Why a document model for interactions?
One pair is one fact, but each pair has different extra knowledge: dosage threshold, age group, monitoring, mechanism. As columns most would be null; as an attribute table every lookup needs joins. A document holds exactly what is known.

Q. What does the validator enforce?
drug_a, drug_b and severity are required, and severity must be low, moderate, high or contraindicated. Anything else is free.

Q. Why store the names alphabetically?
So a pair has one form. Aspirin-Warfarin and Warfarin-Aspirin are the same document, the unique index on (drug_a, drug_b) prevents duplicates, and one lookup answers either order.

Q. What happens when MongoDB is down?
The driver times out after 1.5 s, the API returns 503 and the PostgreSQL transaction rolls back. Dispensing without the safety check is never allowed. We showed it live.

Q. How are severities treated?
Contraindicated is blocked outright. High and moderate must be acknowledged on screen and the acknowledgement is re-checked on the server. Low is informational.

Security and implementation

Q. How is the audit log protected?
Twice. The API connects as pharmacy_app, which only has SELECT and INSERT on it. The tables are owned by pharmacy_owner, which could update, so a BEFORE UPDATE / DELETE / TRUNCATE trigger raises an exception for everyone.

Q. What about SQL injection?
Every query is parameterised ($1, $2) through node-postgres; user input is never concatenated into SQL.

Q. What is the stack?
PostgreSQL 17 and MongoDB 9 installed locally as Windows services, a Node 22 Express API with 21 routes, and a React 18 front end built with Vite and served by the same server.

Q. How do you know it works?
smoke.js runs 24 end-to-end checks against the API: FEFO, rollback without acknowledgement, over-dispensing, contraindication, batch spanning, the race, the outage, EXPLAIN, the audit guard, the validator and the constraints. All 24 pass.

Q. How is the data reset?
node scripts/setup.js drops and rebuilds both databases from the SQL files and the seed in about two seconds.

Q. What would you add next?

Login with roles in the UI, stock alerts on reorder levels, and reading interaction data from a published source instead of a curated seed.